

A Contract-Centered Architecture for Scalable and Manageable Agentic Runtimes

Yaxiao Liu
rootliu@gmail.com

Working preprint | 2026-08-03

1. Abstract

Enterprise AI deployment is a coordination problem across business units, application and AI teams, testing, platform engineering, cloud or server-farm infrastructure, security, operations, and enterprise data governance. Use-case benchmarks can show whether one agent completes one task, but they do not define how rapidly changing capabilities, models, runtime mechanisms, physical capacity, and enterprise data should be owned, changed, admitted, or evidenced together. We present four responsibility objects as shared organizational contracts: Skill is a reusable, versioned business capability and workflow asset; Harness is the runtime compiler and governor; Scaffold is the execution/control boundary and NFR owner; and the data substrate is stack-external, independently CIO-governed semantic and telemetry infrastructure. The runtime core is modeled as $A = \langle S, H, X \rangle$, with the data substrate outside that stack. The central scientific contribution remains a bounded, falsifiable Testable Separability Conjecture. Within a declared operating region, it predicts that logical capability and compatible physical capacity can vary independently when typed closure, complete mediation, effect non-interference, shared-state isolation, resource invariance, and scheduler independence hold. Deterministic gates and trace identity make violations observable, auditable, and attributable; they are not asserted to cause independence. We separate business/use-case evidence from system/runtime evidence and specify protocols for semantic invariance, capacity response, recoupling, Skill training, and data-registry independence. The scale statement of 0 to 100,000 admitted or active agents is a measurable design target, not implementation evidence. The paper contributes an enterprise architecture, testable contracts, and falsification protocols, not completed experimental results.

2. Introduction

Enterprise AI is usually assembled along existing organizational boundaries. A business unit owns a procurement, customer-service, research, or finance outcome. Application and automation developers encode the workflow. AI platform teams change models and agent patterns. Testing and governance teams define release evidence. Cloud, server-farm, site-reliability engineering (SRE), security, and operations teams supply and control execution. The chief information officer (CIO) and domain data stewards govern meaning, access, lineage, and lifecycle across enterprise systems. These groups change different objects at different rates, yet an agent run crosses all of them.

Use-case evaluation alone does not close this deployment gap. A benchmark can establish task success, requirement completion, quality, cycle time, or human correction for a particular Skill. It cannot by itself establish that the capability was admitted under the intended policy, composed safely, bound to compatible capacity, isolated from other tenants, replayable, portable, or affordable at enterprise scale. Conversely, a runtime can meet throughput and availability targets while the Skill produces no business value. Business/use-case evidence and system/runtime evidence are therefore separate and neither substitutes for the other.

The coordination problem becomes sharper as models, agent patterns, tools, and business workflows change faster than infrastructure, security boundaries, or enterprise data standards. Loading every tool description, schema, policy, and source into every request turns organizational growth into prompt growth and obscures decision rights. Adding workers behind the same undifferentiated loop may increase throughput, but it does not explain what was admitted, which path was authorized, what evidence was required, or why a particular execution boundary was selected.

This paper advances a narrower thesis of its own: for enterprise AI, architecture can serve as a shared organizational contract that makes communication, ownership, change cadence, and evidence obligations explicit across business and technology groups. This is not an external quotation. It is the organizing interpretation used here to connect business capability, runtime governance, execution capacity, and enterprise data without requiring one mandatory organization chart.

The architecture is organized around a compact model:

$$A = \langle S, H, X \rangle,$$

where S is a versioned set of Skills, H is the Harness that compiles and governs runs, and X is a pool of Scaffold instances. Skill owns bounded business semantics. Harness owns runtime admission, composition, governance, and evidence capture. Scaffold owns the physical execution and control boundary and its non-functional requirements (NFRs). A stack-external data substrate, independently governed by the CIO or equivalent data authority, supplies semantic joins, governed access, provenance, routing facts, intermediate representations, and integrated business and runtime telemetry.

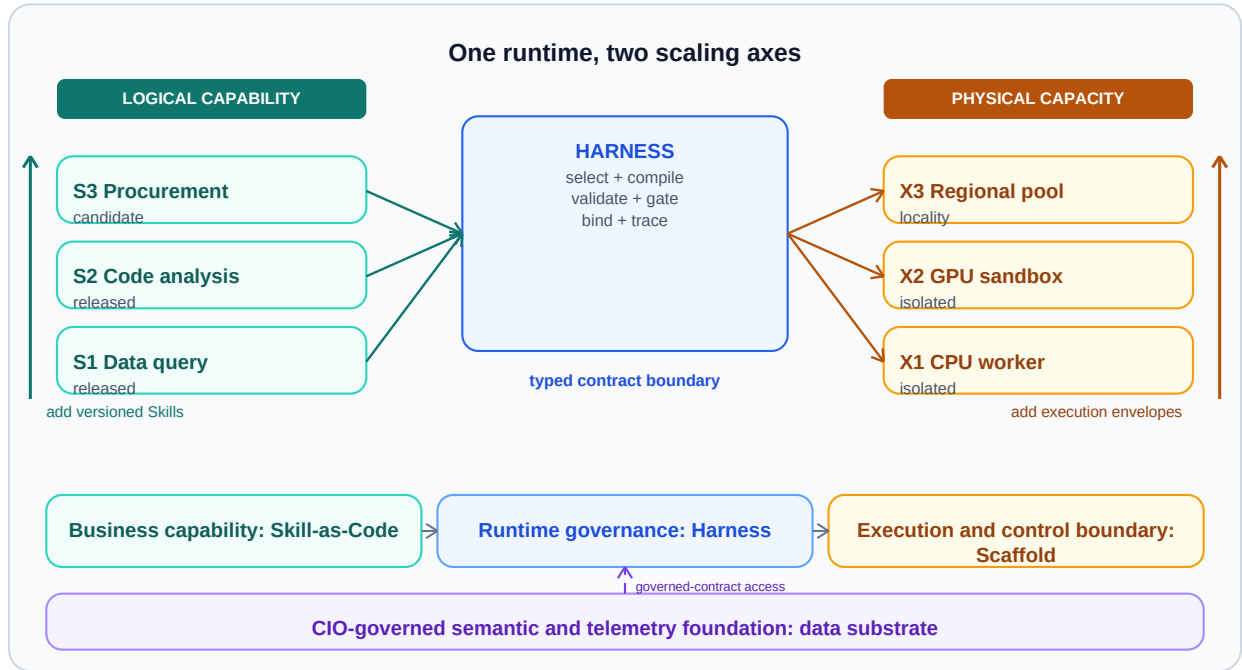


Figure 1. Enterprise responsibility and dual scaling model. Business capability enters as a versioned Skill contract; the Harness admits and binds an activated path; the Scaffold supplies the execution and control boundary. The CIO-governed semantic and telemetry substrate remains stack-external. Logical capability growth and compatible capacity growth are distinct interventions whose separability must be tested inside a declared operating region.

Figure 1 establishes both the responsibility handoffs and the scientific distinction. Logical growth changes what the enterprise can do; physical growth changes how much admitted work it can execute under specified NFRs. The Harness is the contract boundary that makes both interventions inspectable. The paper asks whether that boundary permits logical capability scaling and physical capacity scaling to evolve independently while remaining manageable. The answer is a conjecture, not a theorem: separability is predicted only inside a declared operating region and only when the six conditions in Section 5.3 pass independently preregistered mechanism checks.

2.1 Contributions

This paper makes three primary contributions.

1. We define an enterprise responsibility architecture that separates reusable business capability, runtime governance, physical execution and NFR ownership, and independently governed enterprise data according to stable contracts

and change cadence.

2. We define the Harness as a typed capability-to-capacity binding boundary. Its interface semantics are the admission decision, activated path, authorized effects, binding constraints, and postconditions. Selective activation connects fast-changing Skills to slower-changing infrastructure and data contracts without copying the entire control state into each model context.
3. We preserve and operationalize the Testable Separability Conjecture. A randomized capability-count x Scaffold-count experiment estimates semantic invariance, capacity response, and recoupling interaction; additional protocols test control-plane leakage, composed-path safety, locality and dry-run economics, Skill lifecycle stability, vector-valued training admission, reconstructive reflection, and registry independence.

The contribution is architecture plus falsification methodology rather than synthetic results. Deterministic gates and trace identities make contract semantics inspectable and violations attributable, but they do not prove separability, safety, scale, or business value. Architecture-compatible mechanisms such as context partitioning, derivation-closure orchestration, external-data contracts, locality-aware binding, planning-time dry-runs, an Intermediate Relation, and a train-freeze-monitor Skill lifecycle remain design patterns, secondary hypotheses, or open questions unless separately tested.

3. Origins of the Extensibility Framework

3.1 Organization, Communication, and Architecture

Conway's 1968 observation is the sole historical reference retained in this paper: organizations tend to produce system structures that reflect their communication structures [1]. Enterprise AI makes that observation operationally relevant because one run crosses business ownership, application development, AI runtime governance, testing, infrastructure, security, operations, and data stewardship. A hidden or ambiguous handoff in the organization can become a hidden or ambiguous dependency in the system.

This paper draws a narrower conclusion of its own. Architecture can be used as a shared organizational contract that states who owns an object, what that object promises, what it does not own, how quickly it may change, and what evidence must cross each boundary. The claim is not that every enterprise needs four teams or four processes. It is that business semantics, runtime governance, physical execution, and enterprise data require distinguishable decision rights even when one team implements several of them.

3.2 Ownership and Change Cadence

The four responsibility objects differ most visibly in ownership and expected rate of change. Table 1 is a responsibility model rather than a mandatory organization chart.

Table 1. Enterprise ownership, typical change cadence, and stable contract. One team may implement multiple objects, but their decision rights, stable contracts, and evidence responsibilities remain distinguishable.

Object	Primary enterprise owner	Typical change cadence	Stable contract
Skill	Business product owner with AI/automation developers	Hours to weeks	Goal, typed I/O, effects, tests, evidence, version
Harness	AI platform/runtime and governance teams	Days to months	Admission, activated path, effects, binding constraints, postconditions, trace
Scaffold	Enterprise platform, cloud/server-farm, SRE, security, and operations	Weeks to years	Resource, isolation, locality, identity, attestation, execution
Data substrate	CIO/data authority with domain data stewards	Weeks to years	Semantic access, provenance, routing, lifecycle, intermediate representation

A business workflow or Skill may change within hours, while an isolation boundary, regional topology, or enterprise semantic standard may remain for years. Directly embedding slower-changing infrastructure and data assumptions in every Skill makes both sides expensive to change. The Harness contract instead translates an activated business capability into explicit runtime requirements and binds only accepted work to compatible capacity and governed data access.

3.3 Runtime, Capacity, and External Data Boundaries

The control-plane/data-plane distinction motivates, but does not prove, the proposed separation. Skill registration, release state, contract compilation, policy versions, and placement rules are control decisions. Accepted payload processing and effect execution are data-plane work. Selective activation keeps the full registry and governance state out of each request, while a resolved contract carries only the path, constraints, identities, and evidence obligations needed for that run.

Capability and capacity are therefore treated as different interventions. Adding or revising Skills changes the logical action space. Adding compatible Scaffold instances changes the physical execution space and its capacity response. Shared state, resource-sensitive model behavior, external effects, and scheduler feedback can still recouple them. Section 5.3 states six candidate conditions, and Sections 9.1-9.2 specify the factorial experiment that can count against the separability conjecture.

Enterprise data has a different boundary again. The data substrate is not a fourth runtime layer and not a database hidden inside the Scaffold. It is stack-external, independently CIO-governed semantic and telemetry infrastructure with a typically slower change cadence than Skills. It supplies semantic joins across distributed systems, governed access and policy-aware routing, provenance and lineage, versioned intermediate representations, and integrated model, agent, Harness, business, and infrastructure telemetry. Skills consume its contracts; the Harness mediates access and evidence; the Scaffold enforces locality and network boundaries. On-policy access paths and off-policy indexing or maintenance loops can evolve behind that contract without requiring source-specific context assembly in every Skill.

4. Related Work and Novelty Boundary

Runtime governance and composition.

The five-plane reference architecture organizes production-agent governance into distinct concerns and supplies contemporary support for complete mediation, explicit invariants, and structured evidence [2]. Falsifiable release-gate work further demonstrates a predeclared, machine-checkable invariant discipline over a bounded runtime model [3]. These works motivate enforceable runtime governance, but neither establishes this paper's capability-count $\times$ Scaffold-count separability claim. Composition research shows why the activated path must be the authorization object:

components benign in isolation can become harmful when composed [4]. Our Harness therefore evaluates the path together with its data, effect, identity, and resource bindings rather than treating a set of locally approved Skills as sufficient.

Skills as governed assets.

Skillware supplies a software ontology for persistent, versioned behavioral artifacts and lifecycle continuity [5]. SkillCorpus examines consolidation and evaluation at ecosystem scale [6]. Cross-layer misalignment research shows that a Skill description and its observed behavior can diverge [7]. Together these sources support treating a Skill as a governed asset whose declared interface requires behavioral validation; they do not show that any particular Skill is safe, valuable, or portable. AEVAL separates first-attempt performance from self-correction and separates the executor from the grader [8]. We carry those controls into the release and training protocols because business task success and runtime contract adherence must be reported as distinct evidence.

Harness evolution and structured change.

Harness Handbook identifies behavior localization and editable structure as central problems in harness evolution [9]. Continual optimizer evaluation finds that gains compound only when regression control remains inside the optimization loop [10]. GRACE structures persistent agentic context as a typed semantic graph and verifies edits within the affected neighborhood [11]. These results motivate localized change, regression gates, and explicit intermediate structures. The Harness contract and Intermediate Relation adapt those principles to different objects: capability-to-capacity binding and the coupling between source-oriented and output-oriented registries.

Industry evidence, separated by plane.

Contemporary industry sources illustrate the same evidence split without establishing the proposed architecture. An AWS account of agentic AI in enterprise resource planning supplies business/use-case evidence about workflow transformation [12]. AWS material on AgentOps and distributed hybrid-cloud agentic workloads supplies system/runtime evidence about operational governance and deployment topology [13, 14]. None validates the complete Skill-Harness-Scaffold architecture, P1, the 0 to 100,000 design target, or the narrower thesis that architecture can serve as a shared organizational contract.

Responsibility boundary and novelty.

The paper does not score named products or infer the absence of undocumented capabilities. Table 2 instead states the ownership and non-ownership claims of the proposed architecture.

Table 2. Responsibility boundaries in the proposed architecture. The table defines handoffs and non-ownership; it is not a completeness rating of products or organizations.

Boundary	Business semantics	Runtime governance	Physical execution	Enterprise data
Skill	owns	declares requirements	does not own	consumes governed contracts
Harness	compiles activated capability	owns	binds but does not supply	mediates access and evidence
Scaffold	does not interpret	exposes enforceable facts	owns	enforces locality and network boundary
Data substrate	preserves domain meaning	supplies policy/provenance facts	does not schedule	owns semantic integration and routing

The novelty claim is deliberately bounded. This paper does not propose a new model architecture, scheduler algorithm, data ontology, or universal governance taxonomy. Its enterprise contribution is a shared organizational contract across four responsibility objects. Its scientific contribution is the precise Harness boundary, six candidate causal conditions, and protocols that can reject separability inside a declared operating region. Architecture definitions and cited mechanisms are not implementation evidence for P1, the scale target, semantic-join effectiveness, portability, safety, or business value.

5. Model and Assumptions

5.1 The Three Runtime Objects

Skill.

A Skill s in S is a versioned declaration of task capability. Its minimum interface is

$$s = \langle g, i, o, p, e, v \rangle,$$

where g describes the goal and applicability conditions, i and o are typed input and output schemas, p contains preconditions and policy tags, e declares externally visible effects, and v identifies the validated version. A Skill can include model instructions, tool adapters, graph fragments, examples, and tests, but these assets are implementation details behind the contract. A Skill is operation-closed when every external effect it can request is declared and testable through its interface.

Collection-scale evidence suggests that operation closure is not what current practice provides. The SkillMD-138K snapshot released with the Skillware ontology [5] contains 138,133 content-deduplicated SKILL.md records drawn from 20,556 repositories, and two of its reported measurements bear on closure. First, frontmatter is nearly universal (136,380 records, 98.73%), which the authors read narrowly as evidence that a structured metadata envelope is a dominant visible convention, explicitly not that its fields are validated or that activation semantics agree across agent systems. Second, a lexical detector finds an explicit path token into scripts/, references/, or assets/ in 32,069 records (approximately 23.2%); the authors note that this detector misses implicit dependencies, so the figure is a lower bound on packaged artifacts rather than a census. Even read as a lower bound, it indicates that a large share of the corpus is natural-language text with no declared deterministic execution component. Neither figure speaks directly to the fields operation closure requires. A frontmatter envelope establishes that a name and a description are conventional, which corresponds to the goal field g ; it does not establish a typed output schema o , a declared effect set e , or a version identity v that a gate could reject independently. The corpus statistics therefore bound what can be claimed: a metadata envelope is the prevailing convention, whereas the typed and independently rejectable interface this architecture depends on is not visible at collection scale. That gap is the reason the Harness must enforce closure as an admission condition rather than assume it of registered Skills. Whether closure can be retrofitted onto existing Skills at acceptable cost is an empirical question we do not settle here.

Harness.

The Harness H is the runtime compiler and governor. Given a request q , identity and policy context u , and a Skill registry S , it produces a candidate execution graph D , validates the graph, binds accepted nodes to Scaffold instances, and emits a trace:

$$(q, u, S) - [H] \rightarrow (D, C, b, \tau).$$

Here C is the resolved Harness contract, b is a binding from graph nodes to Scaffold instances, and τ is the audit and replay trace. Selection and graph construction may use an LLM and are therefore probabilistic. Admission, schema validation, policy evaluation, budget enforcement, effect authorization, and trace creation must be deterministic for a fixed contract and policy snapshot.

Scaffold.

A Scaffold instance x in X is a physical execution envelope:

$$x = \langle r, z, l, a \rangle,$$

where r describes available resources, z describes isolation and trust zone, l describes data and network locality, and a describes attestation and runtime identity. A Scaffold may be a process sandbox, container, virtual machine, browser session, graphics processing unit (GPU) allocation, or remote execution pool. It exposes resource facts and execution primitives, not task semantics. As the execution and control boundary, it owns these NFRs: performance, reliability, availability, observability, manageability, security, isolation, portability, elasticity, and capacity evolution.

5.2 The Harness Contract

The resolved contract for one run is

$$C = \langle I, O, G, B, E, T \rangle.$$

I and O are the run-level typed inputs and outputs. G is the activated Skill graph. B contains resource, time, token, cost, concurrency, placement, and isolation constraints. E contains authorized effects and evidence obligations. T contains policy, model, registry, data-snapshot, and trace identities needed for replay. The boundary's interface semantics are defined by five observable components: the admission decision over C, the activated path in G, the authorized effects in E, the binding constraints in B, and the postconditions encoded by O and its evidence obligations. This tuple is intentionally operational: each field must be serializable, inspectable, and independently rejectable.

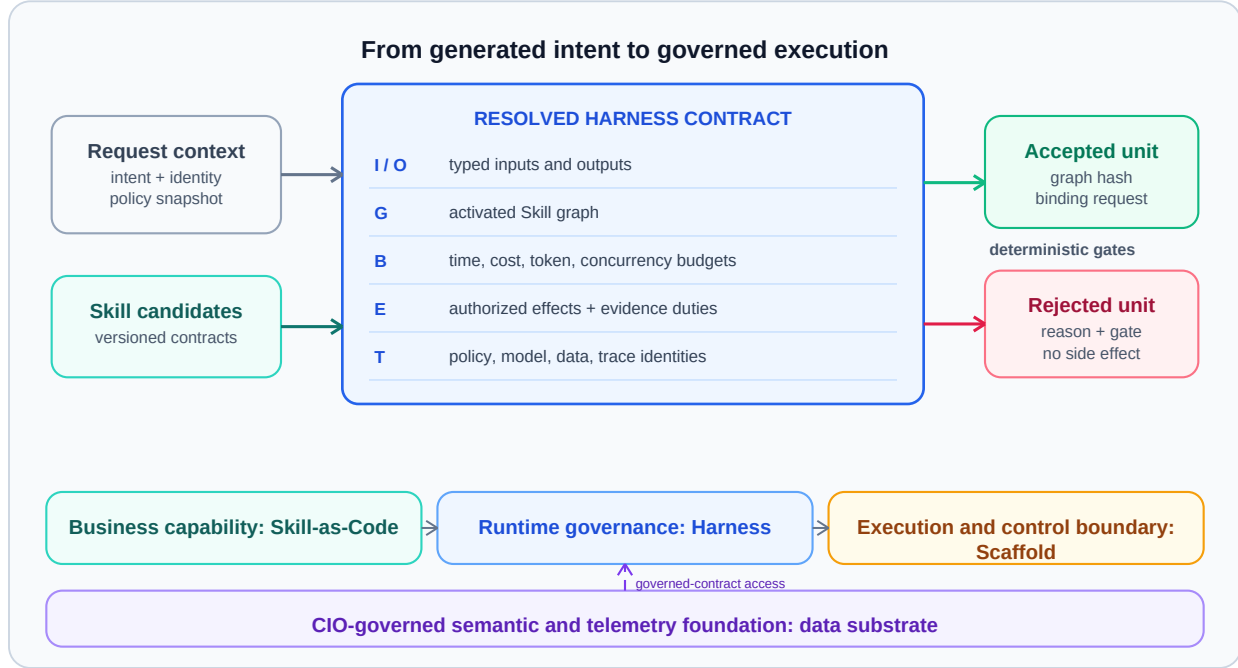


Figure 2. Anatomy of a resolved Harness contract. A probabilistic plan becomes executable only after schemas, graph structure, budgets, effects, evidence obligations, and trace identities have passed deterministic gates. Manageability resides in the transition from generated intent to a typed, auditable execution unit.

Figure 2 makes the Harness boundary concrete. A useful shorthand is: probabilistically written, deterministically and auditably executed. The phrase does not imply that execution outcomes are deterministic; external services and models remain variable. It means that the authorization decision, declared effects, selected versions, and evidence requirements can be reconstructed from the recorded contract.

5.3 Conditional Decoupling

The separability conjecture is scoped to a declared operating region Ω . A study of Ω fixes the workload class; model, tokenizer, and policy versions; external-service contracts; admissible resource classes; and scheduler regime. A change outside those bounds is a change to the operating region, not an isolated ΔS or ΔX intervention. Within Ω , the conjecture names six causal conditions. They are hypotheses about when semantic behavior and capacity response can vary independently, not a proof that the architecture guarantees independence.

1. Typed closure. The activated operation and effect surface resolves to versioned typed input, output, and effect declarations; operations or effects outside that surface are rejected before execution.
2. Complete mediation. Every Scaffold, tool, and external-system invocation traverses an instrumented Harness binding point; there is no alternate execution or call channel outside that boundary.

3. Effect non-interference. Effect-labelled data or control flow from one activated path can reach another only through a dependency declared in their contracts; undeclared cross-path read, write, and control channels are absent.
4. Shared-state isolation. Every mutable store is assigned to a declared namespace, snapshot, or synchronization protocol, and cross-partition access is possible only through a declared shared-state dependency.
5. Resource invariance. Scaffold count, topology, and identity have no channel into semantic inputs: across compatible Scaffold arms, model, tokenizer, policy, numerical-mode, prompt and tool payload, timeout and retry, and sampling configurations are unchanged except for declared binding fields and resource-only telemetry.
6. Scheduler independence. Scheduler-labelled signals are confined to declared ordering and resource-dependency fields and have no undeclared flow into admission, path-selection, effect-authorization, binding-constraint, or postcondition logic.

Before outcome collection, each study must preregister a separate mechanism-level acceptance criterion and its instrumentation for every condition: (i) typed closure passes only if every activated operation resolves to the declared versioned types and all preregistered undeclared-operation and undeclared-effect probes fail closed; (ii) complete mediation passes only if every observed invocation carries a valid Harness binding and negative probes expose zero unbound call paths; (iii) effect non-interference passes only if information-flow traces and cross-path probes expose zero undeclared read, write, or control edges; (iv) shared-state isolation passes only if every mutable access maps to its declared namespace, snapshot, or synchronization protocol and probes expose zero undeclared cross-partition accesses; (v) resource invariance passes only if configuration and payload hashes for the semantic-input fields listed above are equal across compatible Scaffold arms and resource-labelled taint reaches none of those fields; and (vi) scheduler independence passes only if scheduler-perturbation traces expose zero scheduler-labelled flows outside the declared ordering and resource-dependency fields. These condition criteria are evaluated without semantic task scores, interface-semantic outcomes, throughput, latency, or factorial interactions. An outcome leaving its margin does not itself fail a condition, and passing all six condition checks does not establish separability.

Under these candidate conditions, adding s_{new} to S is predicted to leave capacity response unchanged for existing workloads, while adding compatible x_{new} to X is predicted to preserve semantic behavior and the five interface semantics. The Harness contract is the candidate stable interface. Shared mutable state, dynamic policy, nondeterministic external effects, resource-sensitive model behavior, cross-Skill interference, and scheduler feedback can re-couple the axes even when declared interfaces remain unchanged; the operating region and evaluation protocol must therefore expose these factors rather than assume them away.

Testable Separability Conjecture. Within a declared operating region Ω , if typed closure, complete mediation, effect non-interference, shared-state isolation, resource invariance, and scheduler independence meet their independently preregistered mechanism-level acceptance criteria, then interventions on Skill capability count and compatible Scaffold count are predicted to be separable: Scaffold changes preserve preregistered semantic behavior and the five interface semantics, while Skill changes preserve the capacity-response function for existing workloads.

The six conditions identify concrete recoupling channels without asserting necessity. Lack of typed closure permits undeclared behavior; incomplete mediation permits direct binding; interfering effects and shared state transmit changes between paths; resource observations can enter semantic inputs; and scheduler signals can enter task-control logic. A system may compensate for one weakness through a mechanism outside this model, so observing separability after an apparent condition violation would refine the conjecture rather than establish a converse.

Deterministic gates form part of a separate observability requirement. Gates make the admission decision, activated path, authorized effects, binding constraints, and postconditions reproducible for fixed recorded inputs. Trace identity completes that requirement by linking those decisions to the Skill, model, policy, data snapshot, binding, and effects involved. Violations become detectable, auditable, and attributable through this requirement, which is not a causal generator of independence. P1 is rejected in the tested operating region if all six independently evaluated condition criteria pass but preregistered semantic equivalence or non-inferiority fails, capacity response departs beyond its margin, or a capability-count $\times$ Scaffold-count interaction exceeds its margin. A non-significant interaction alone does not establish independence.

P1 is the core structural conjecture. P15 (dual-subgoal convergence in Section 8.5), P16 (reconstructive reflection sufficiency), and P17 (Intermediate-Relation decoupling in Section 8.2) address training and data subsystems. Table 3 separates claim type from current evidence status.

Table 3. Claim types and evidence status. "Proposed experiment" denotes a protocol, not a completed study in this paper.

Object	Claim type	Scoped claim	Evidence status
Enterprise-runtime responsibility architecture	Design proposal	Four responsibility objects separate business capability, runtime governance, execution/NFR ownership, and enterprise data governance through stable contracts and change cadence	Architectural synthesis; cited motivation; no implementation, adoption, or empirical validation
Shared organizational contract	Interpretive thesis	Architecture makes communication, ownership, change cadence, and evidence obligations explicit across enterprise groups	This paper's organizing thesis; no external or empirical validation
P1	Proposition	Testable Separability Conjecture inside Omega	Analytic argument; proposed experiment (Section 9.1-Section 9.2)
Path safety / mediation	Proposition	Complete mediation enables path-level admission checks	Analytic argument; cited external evidence; proposed experiment (Section 9.3, Section 9.5)
P15	Hypothesis	Output-plus-process gates improve discrimination, attribution, or training	Cited external evidence; proposed experiment (Section 9.8)
P16	Hypothesis	Pinned summaries support use-case-relative reconstruction	Proposed experiment (Section 9.9)
P17	Proposition	Registry schemas/modules remain independent under declared changes	Analytic argument; cited external evidence; proposed experiment (Section 9.9)
Dry-run / locality	Hypothesis	Planning cost is offset by avoided work or movement	Proposed experiment (Section 9.6)
Skill lifecycle / training	Design pattern	Train, validate, freeze, monitor, and roll back Skills	Cited external evidence; proposed experiment; future work
Higher-order memory	Open question	Structures beyond the contract-relevant IR	Future work

Table 4. Two required evidence planes for enterprise adoption. Each plane answers a different decision question; neither plane substitutes for the other.

Evidence plane	Primary decision question	Representative evidence	Does not establish
Business/use-case	Does the versioned Skill produce the intended business outcome for its declared users and workflow?	Task and requirement completion, output quality, cycle time, human correction, adoption, and outcome-specific benefit or loss	Runtime admission integrity, isolation, capacity, availability, portability, or cost behavior
System/runtime	Does the admitted path execute within its declared contract and enterprise NFR envelope?	Gate and trace integrity, path and effect conformance, throughput, active-agent capacity, latency, saturation, failures, isolation, availability, backpressure, and cost	Usefulness, workflow fit, user acceptance, or business value

The business owner and Skill team require the first plane; runtime governance, platform, SRE, security, operations, and data authorities require the second. A release decision must name the evidence required from both planes and must not use success in one as a proxy for the other.

6. Harness Manageability

6.1 Control Plane and Data Plane

The control plane owns Skill registration, contract compilation, policy versions, placement rules, and release state. The data plane carries request payloads, executes accepted nodes, moves authorized evidence, and emits traces. The planes interact through references and resolved contracts rather than by copying the full control state into every model prompt.

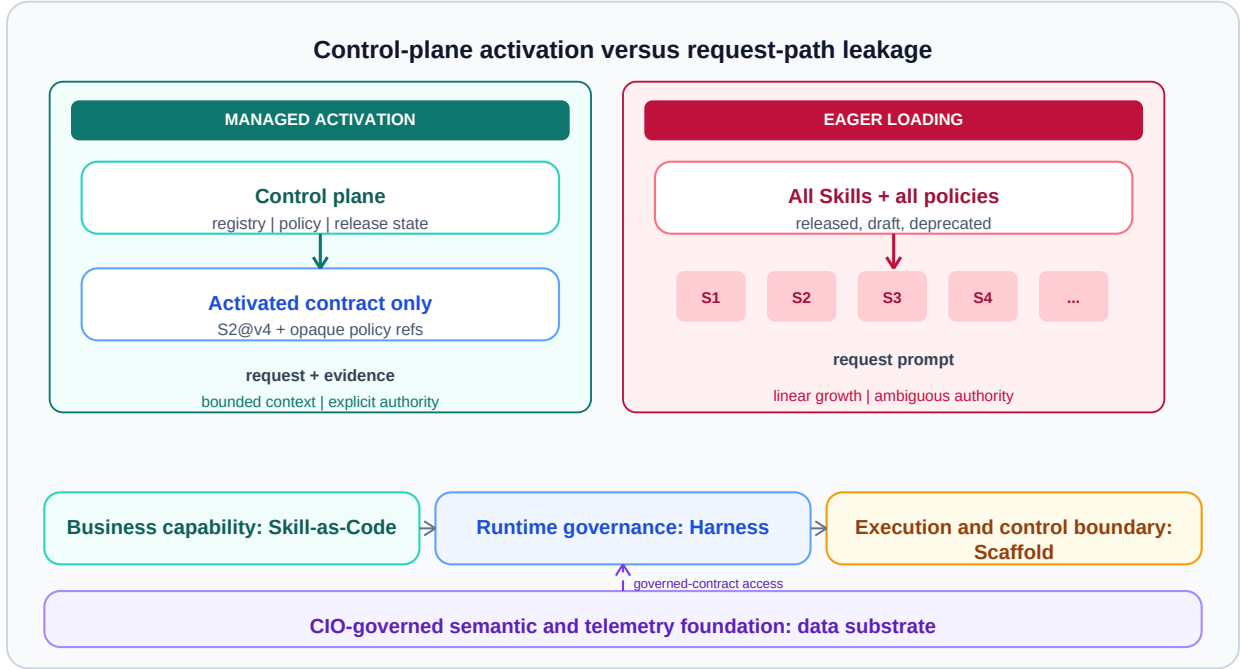


Figure 3. Control-plane separation versus control-state leakage. In the managed design, the request receives only activated contracts and opaque version references. In the leaking design, every Skill schema and policy is injected into the request path. Selective activation is intended to prevent logical scaling from becoming linear prompt growth and to limit exposure of governance metadata.

Figure 3 highlights a failure mode that is easy to mistake for flexibility. Loading every Skill, application programming interface (API) schema, and policy example into each request leaks control-plane state into the data plane. Besides token cost, the leak creates ambiguous authority: a description visible to the planner may look executable even when its release or policy state has changed. The Harness should first select a small candidate set from registry metadata, then materialize only the accepted versions and constraints needed by the run. Whether selective activation actually bounds context growth and preserves throughput is an empirical question tested by the shared factorial experiment, not an architectural guarantee.

6.2 Path-Aware Invariants

Per-Skill approval is necessary but insufficient. Let $G_q=(V_q,E_q)$ be the activated graph for request q , with nodes representing Skill operations and edges representing data or control dependencies. The Harness evaluates invariants over the path:

$$\text{Accept}(G_q)=\text{conjunction over } k \text{ } \phi_k(G_q,u,B,E,T).$$

Examples include separation of duties, data-residency continuity, prevention of read-then-publish paths, maximum cumulative privilege, and mandatory human approval before an irreversible effect. A path can violate an invariant even when each node is locally allowed. Runtime checks must also cover dynamically inserted nodes and retries, since they change the effective graph.

6.3 Audit, Replay, and Reversibility

An audit trace should distinguish three questions: what the planner proposed, what the deterministic gates accepted, and what the bound Scaffold actually executed. Conflating these stages makes incident review nearly impossible. At minimum, the trace records request and principal identity, candidate and activated Skills, contract versions, policy decision inputs, graph hash, binding decisions, external calls, evidence artifacts, and final effects.

Replay has levels. Decision replay recomputes gate outcomes from recorded inputs. Plan replay reruns planning under the recorded model and prompt version, accepting that stochastic output may differ. Execution replay re-executes against pinned data or service snapshots when available. The runtime should declare which level it supports rather than promising exact replay across mutable external systems.

Manageability also requires a stop path. Skill releases can be disabled, policy versions rolled back, bindings revoked, and in-flight graphs cancelled at Harness checkpoints. These mechanisms turn a trace from passive observability into operational control.

7. High Cohesion, Loose Coupling, and Context Partitioning

7.1 First Principle: The Agent's Job Is Organizing Context

For a fixed model, fixed sampling or decoding configuration, fixed runtime tool configuration, and fixed external state, a single LLM inference varies with its context input. Within that controlled setting, context is the variable an agent loop directly organizes for the next inference step. Across arbitrary agent runs, the model, decoder, runtime tools, and external systems may also vary, so context is not literally the only variable. Planning, data retrieval, tool invocation, and memory writes nevertheless remain decisions about what to place in the next context and what to keep external to it.

This observation has a direct consequence for performance under serving systems that expose prefix-cache reuse (for example, vLLM or SGLang). Cache identity requires more than semantically equivalent prompts: the model and tokenizer versions, system and policy metadata, tool serialization and ordering, and every prefix token must remain fixed. If a single token changes in the middle of the prefix, all subsequent positions must be recomputed. The reusable part is not "the context that changed slightly" but "the context whose prefix is frozen token-by-token with changes only at the tail." Heterogeneous tools or goals can therefore reduce cache reuse and introduce irrelevant context, but this is one candidate mechanism for long-horizon degradation rather than a complete explanation.

7.2 Context Partitioning by Tool-Set and Data Source

The corresponding design heuristic is: place version-pinned stable material (system prompt, policy metadata, serialized tool schemas, and fixed memory) at the front in deterministic order; place per-turn task content at the tail. Under a prefix-caching serving stack, a longer token-identical prefix creates a larger reuse opportunity. Stable semantic content alone is insufficient if serialization, ordering, tokenizer, or model versions differ.

One useful axis for partitioning context is the tool-set. A sub-agent that repeatedly uses the same version-pinned set of tools and Harness contracts can maintain a stable prefix when its prompt renderer is deterministic. Conversely, switching among heterogeneous tool-sets may rewrite that prefix and dilute attention. Stable tool membership therefore helps prefix stability but does not guarantee it; the system must measure token identity and cache hits directly. A related heuristic is to divide sub-agents by minimally overlapping tool-sets when doing so preserves task semantics.

The secondary axis is data. Detail such as raw tables, full document text, and verbose intermediate results should not flow between sub-agents as context. Instead, detail is written to the data subsystem (memory or lake), and only summarized context (conclusions, handles, key constraints) is passed forward. Downstream sub-agents retrieve detail on demand via semantic join, not by carrying it in their context. Under explicit interface assumptions that summaries and handles are size-bounded, detail remains external, and downstream retrieval traffic and cost are accounted for separately, the handoff payload changes from $O(\text{full context})$ to $O(\text{summary})$. This is a conditional communication-size claim, not an unconditional reduction in total system work.

7.3 Four Mechanisms for High Cohesion and Loose Coupling

The partitioning principles above yield four concrete design mechanisms that span all three runtime layers:

1. Tool-set cohesion (Skills layer). A sub-agent should use one version-pinned tool-set and deterministic prompt serialization throughout a run. This increases the opportunity for prefix stability and KV-cache reuse, which must still be measured. The anti-pattern is an agent that changes tool schemas, ordering, or policy metadata across successive turns without recording the resulting cache discontinuity.
2. Minimally overlapping partitioning (parallelism). When a task is decomposed into parallel sub-agents, their tool-sets should be as disjoint as task semantics allow. Disjoint tool declarations are only a proxy for a smaller shared surface: compute pools, locks, external services, authorization backends, data dependencies, and scheduler state may still create contention or correlated decisions and must be measured separately.
3. Summarized-context handoff (loose coupling). Sub-agents pass only summarized conclusions and handles to memory, not full context. The receiving sub-agent retrieves detail on demand from the data subsystem. This creates a loosely-coupled sequential relationship: dependencies flow through thin summary interfaces, while detail is absorbed by the memory layer. The data subsystem sits outside the three-layer stack, so this absorption does not introduce coupling into the runtime core.
4. Seed-affinity placement (Scaffold layer). Each type of sub-agent has a derivation seed, an image with pre-warmed tools, dependencies, and cache. Same-seed sub-agents are co-located to reuse image layers, tool processes, and KV-cache hot regions (affinity); different-seed sub-agents are spread across resource domains to reduce contention (anti-affinity). This maximizes physical locality within a type and minimizes cross-type interference.

The parallelism argument of mechanism 2 can be stated as a calibration hypothesis. If a fraction s of a workload remains serial, Amdahl's law caps the speedup at $1/(s+(1-s)/N)$ for N parallel sub-agents. We hypothesize $s=s_0+\text{overlap coefficient}\rho+\epsilon z$, where ρ measures tool-set overlap and z collects independently measured shared-compute, lock, service, authorization, data, and scheduler coupling. We likewise hypothesize $E=E_0-\mu c_{\text{bar}}$ for orchestration efficiency E and average handoff cost c_{bar} . Neither linear relation is implied by the architecture; coefficients, residual structure, and useful operating ranges must be estimated under Section 9. Tool partitioning raises the parallelism ceiling only if reductions in ρ are not offset by the other terms.

The four mechanisms can be composed: mechanisms 1 and 4 target intra-agent locality, while mechanisms 2 and 3 target inter-agent coupling. Tool-set partitioning is an observable design variable across the Skills, orchestration, and Scaffold-placement layers, but the proposed effects remain hypotheses until token identity, cache reuse, contention, handoff cost, and end-to-end behavior are measured together.

7.4 Derivation-Closure Orchestration

The four mechanisms describe how a partition should look once chosen. They do not say how the runtime arrives at a partition for a request whose required tools and sources are not known in advance. We close that gap with a rule we call derivation closure: each admitted sub-agent contract is closed over named operations, evidence sources, input/output and handoff schemas, effects, and budgets. Any reference outside that admitted closure is not satisfied in place but becomes the admission condition for deriving a new sub-agent.

Bounding the output range.

Fixing those admitted operations, evidence sources, schemas, effects, and budgets bounds the sub-agent's authorized action-and-evidence range because each contract dimension is enumerable at admission time. It does not bound every natural-language token sequence the model might emit, nor does it guarantee semantic correctness. This is the operational reading of high cohesion: the sub-agent is cohesive not merely because its prompt is focused but because its executable authority and evidence duties are contractually closed. A request that would widen that closure is precisely the event the Harness must observe rather than absorb silently.

References as derivation conditions.

When a sub-agent encounters a needed search tool or data source that its contract does not cover, granting it inline would erode both the frozen prefix and the closure property. Instead the reference is recorded as an unmet dependency and triggers derivation: the Harness resolves a fresh contract whose tool-set and data-set cover that dependency, seeds a new sub-agent from the matching template, and binds it as a child. Capability growth within a single request therefore proceeds by adding bounded sub-agents rather than by enlarging any existing one. Each derivation event is an ordinary contract compilation and passes the same deterministic gates of Section 5.3, so the request-level graph remains typed and replayable no matter how the partition was discovered.

The main agent as a satisfaction controller.

The main agent does not execute task tools. Its function is to summarize the outputs of its sub-agents and propose whether the run may terminate. For each output sub-domain d , the contract binds a versioned verifier V_d , its evidence inputs, and a threshold τ_d before execution. The controller performs a semantic join across returned summaries, applies top-k selection to retain the best-supported evidence per field, and reports the satisfaction ratio of the declared output sub-domains:

$$\text{sat}(\text{output schema}) = (1 / (|\text{dom}(\text{output schema})|)) \sum \text{over } d \text{ in } \text{dom}(\text{output schema}) \text{ of} \\ \text{indicator}[V_d(\text{join}_k(\text{summaries}_d) \text{ at least } \tau_d)].$$

When every V_d is deterministic or version-pinned with a preregistered error tolerance, the ratio can participate in a contract-level termination predicate. When a verifier is model-based, mutable, or uncalibrated, the ratio is an estimate and cannot by itself authorize termination; the contract must require an independent deterministic gate or named human approval. Subject to that distinction, the loop continues while the ratio is below its target and some unsatisfied sub-domain still admits an untried derivation. Unsatisfied sub-domains identify what is missing, and unmet references supply derivation candidates, so the controller re-invokes existing sub-agents or derives new ones until the target is certified, the derivation budget is exhausted, or no candidate remains. The last two cases are typed outcomes whose unsatisfied sub-domains and verifier versions appear in the trace.

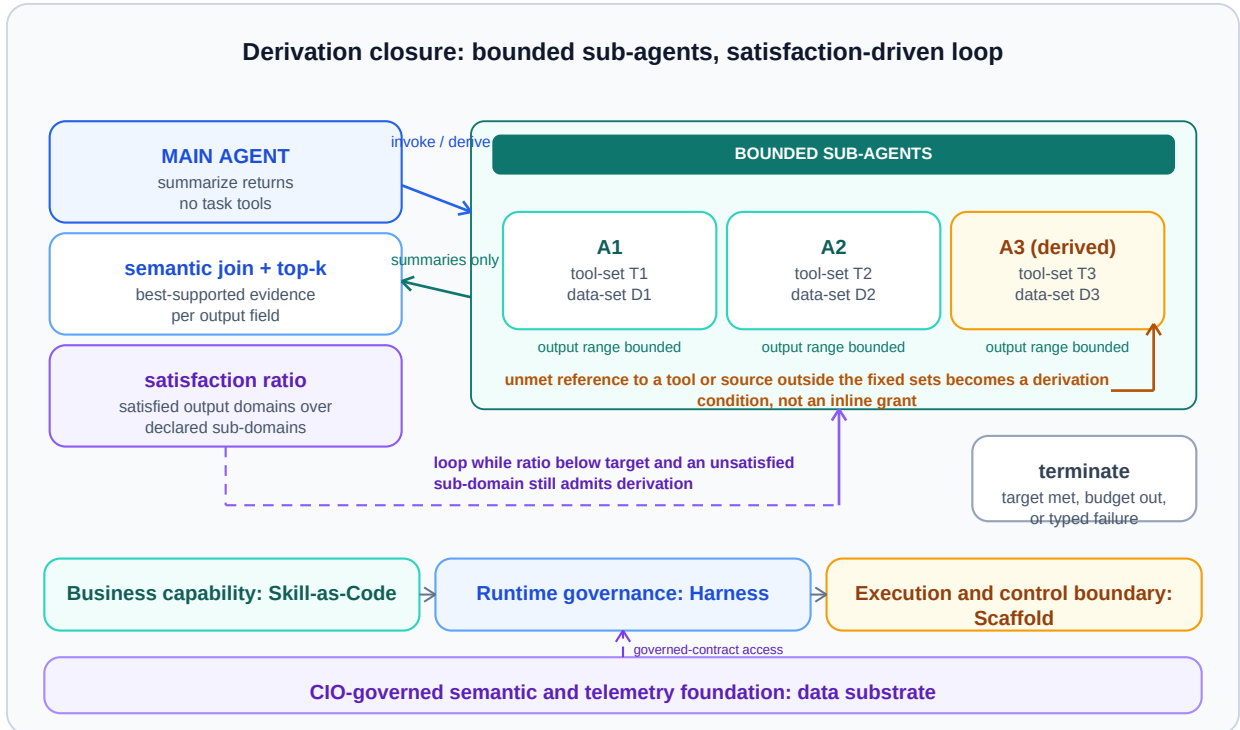


Figure 4. Derivation-closure orchestration. Each sub-agent holds an admitted tool-set and data-set; references outside those sets become conditions for deriving a new sub-agent rather than being granted inline. The main agent summarizes returns, performs a semantic join with top-k selection, and reports a satisfaction ratio computed by version-bound per-domain verifiers. Only deterministic or calibrated, version-pinned verifier results may participate directly in termination; other scores remain estimates requiring an independent gate.

Figure 4 shows the loop. Three properties are testable. First, version-bound verifier results can move termination from an implicit model judgment into an inspectable contract predicate, subject to verifier validity. Second, because derivation is the only way to widen tool or data access, the trace is a complete record of admitted and activated capabilities, not a claim about every capability the task counterfactually needed. Third, the per-sub-domain structure aligns orchestration with the training gate of Section 8.5, allowing the same declared decomposition to be evaluated at training and run time.

7.5 Contract-Compatible Mechanisms

The Harness contract can encode which tools a sub-agent may invoke (mechanism 1), declared independence candidates (mechanism 2), summarized handoff schemas (mechanism 3), and seed templates (mechanism 4). That makes the mechanisms implementable and auditable through one boundary, but does not guarantee prefix stability, physical independence, low handoff cost, or favorable placement. A planning-time dry-run can probe closure, estimate token-prefix stability, identify shared resources, and check capacity before committing an external effect; Section 9 tests whether the resulting predictions are useful.

8. Architecture-Compatible Mechanisms

8.1 External Data as a Contract Instantiation

External data should not be a privileged side channel into the agent. It is a Harness-contract instantiation with explicit authentication, authorization, schema, snapshot, provenance, freshness, residency, and evidence obligations. A data Skill may discover or query assets, but the resolved contract determines what can be fetched and what evidence must accompany the result.

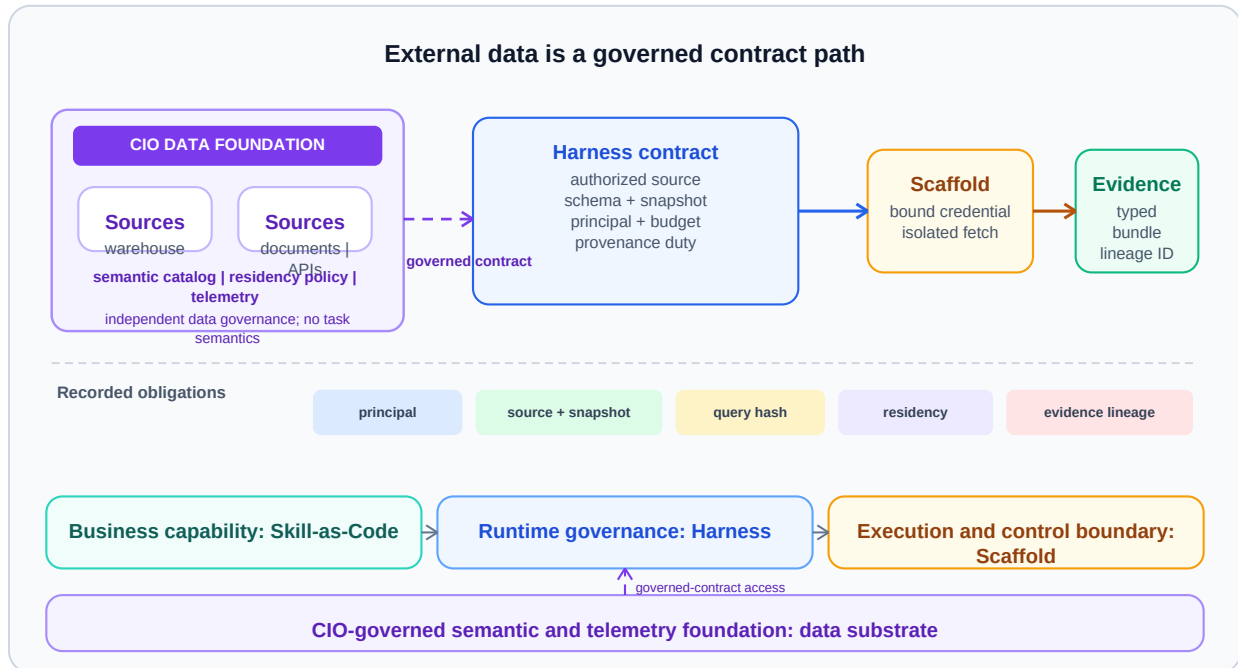


Figure 5. External-data access as a Harness-contract instantiation. Source discovery and query planning remain semantic operations, while credentials, snapshots, provenance, residency, and evidence packaging are resolved and enforced at the contract boundary. Data governance composes with agent execution without embedding infrastructure credentials or source-specific policy in the Skill.

Figure 5 shows how this boundary preserves both portability and accountability. The same analytical Skill can run against different warehouses or document stores when their adapters satisfy the contract. Conversely, the same data source can support multiple Skills without learning their task semantics. Evidence returned to the model is a typed bundle carrying source and snapshot identity, not an unqualified text fragment.

Hypothetical AI4Science walkthrough.

Consider a request to screen perovskite candidates for a target band gap near 1.3 eV using recent literature and to produce a cited, reproducible report. This is an architecture walkthrough, not an implemented case study or empirical validation. Its explicit chain is intent -> contract compilation -> path gate -> binding -> trace -> typed failure -> recoupling. At intent, the request fixes the band-gap target together with recency, citation, and reproducibility constraints. During contract compilation, the Harness activates literature-review, materials-screening, and report-generation Skills and closes their typed inputs, outputs, effects, and evidence obligations. The path gate checks data authorization, institutional HPC identity, a network allowlist for literature and materials databases, and effect closure before any fetch or computation. Binding resolves approved databases, a microVM for scripts, and a compatible GPU or HPC Scaffold without exposing infrastructure credentials to the Skills. The trace pins Skill and model versions, literature and materials-data snapshots, binding and scheduler decisions, screening-script identity, and citation evidence. A missing authorization, ungrounded citation, unit anomaly, or absence of compatible HPC is returned as a typed failure, rather than repaired through an undeclared path. Finally, recoupling is visible if a shared HPC quota, scheduler feedback, or resource-sensitive scientific output makes a Scaffold change alter the admission decision, activated path, authorized effects, binding constraints, postconditions, or scientific result. The example therefore identifies where semantic invariance can fail; it does not establish that the conjecture holds.

8.2 From Query Plan to Intermediate Relation

The stack-external data substrate can pre-summarize sources through an off-policy maintenance loop while the Harness consults those summaries through governed contracts. Here we make the "plan" step concrete as an explicit, auditable record we call the Intermediate Relation (IR), situated between two orthogonal registries.

The Intermediate Relation.

The Data Wiki holds what the data is: per-source semantic summaries produced by the off-policy loop (a natural-language summary, format, retrieval keywords, and cross-source relations). For complex reports whose semantics an agent cannot reliably guess, such as multi-sheet spreadsheets with hidden formulas and drifting definitions, entries are built under human-agent collaboration and tested by reconstructive reflection. P16 is deliberately use-case-relative: before summary construction, the evaluator preregisters the target report family, reconstruction fields, and report-structure descriptor. Reports are separated into training, validation, and held-out sets; the reconstruction agent receives no source credentials or original tables; summaries may be revised from training and validation failures only; and the held-out set is inspected once for the final comparison. The resulting reconstruction rate tests whether a version-pinned summary is sufficient for that preregistered use case, not whether it contains every possible meaning of the source. The Theme Wiki holds what a verified output looks like: reusable templates for output types (reports, charts, decks, analytical write-ups, data-model programs) registered only after the corresponding output sub-domain has been trained and validated under the lifecycle of Section 8.5. The two registries are designed to be orthogonal: a data summary should not change merely because output demand changes, and an output template should not bind to one source.

Orthogonal Wikis.

The IR is the sole explicit coupling point between them. For a given theme it records which source summaries are hit by semantic join, packaged with six contextual dimensions:

$$IR: theme \rightarrow \langle \text{Sigma}(src_i), 5W1H \rangle,$$

where When states validity and update cadence of the data for this theme; Where states the access-domain boundary (intranet, extranet, public internet); Who states data ownership and whether this agent or theme is authorized; What states the business semantics of the fields; Why states the integration logic, that is, why these sources belong together and the reason behind the semantic join; and How states the physical access path (database, lakehouse, JSON, or a Harness tool). The 5W1H dimensions do not bypass semantic join; they are metadata that must accompany its hits, because a hit without them may be stale (no When), unauthorized (no Who), or unreachable (no How). With the IR explicit, the contract mapping becomes: intent determines a theme, the IR resolves its source set and 5W1H record, the

Harness validates timeliness, authorization, and access, and only then issues data fetches.

Decoupling claim and contribution boundary.

P17 distinguishes three change classes: an entry-content update within an existing registry schema, a registry schema or module update, and cross-registry propagation. The proposition concerns schema/module independence, not immutability of entries or the IR: a compliant one-sided entry change should not force a schema or module change in the other registry. Some source or output changes will legitimately modify entries in both a registry and the IR, and a genuinely new semantic type may require a local schema extension; the falsification question is whether that extension needlessly propagates across the registry boundary. Protocol 9.9 tests all three classes. Concurrent work supports the mechanism while differing in target. Scoped verification of evolving agentic context [11] keeps a persistent instruction as a typed graph and validates each proposed edit only within the local neighborhood of the nodes it touches, reporting a substantial reliability gain over an otherwise identical flat-text baseline. That result is evidence for the general principle that explicit structure can confine a change's blast radius. The IR applies the principle to a different boundary: it structures the coupling between a data registry and an output registry, so the property under test is mutual schema/module independence under one-sided change. This subsection proposes no new retrieval or data-integration algorithm. Its contribution is the systems-level reorganization of information the data substrate already holds into an explicit, auditable relation layer; its retrieval quality and semantic-join effectiveness remain unproven.

8.3 Parallelism, Locality, and Planning-Time Dry-Run

Once the Harness has a graph, it can expose parallelism that a monolithic prompt obscures. Independent nodes may execute concurrently; dependent nodes remain ordered by graph edges and effect constraints. The Scaffold pool contributes locality and capacity facts. The binding algorithm can then co-locate data-intensive nodes, isolate high-risk effects, and reserve scarce accelerators only for nodes that declare them.

A planning-time dry-run resolves graph structure, candidate bindings, budget envelopes, policy decisions, and expected evidence without committing external effects. It answers operational questions before execution: Which data regions will be crossed? Which nodes can run in parallel? Which irreversible effects require approval? Is the predicted cost within budget? Are all required Scaffold types available?

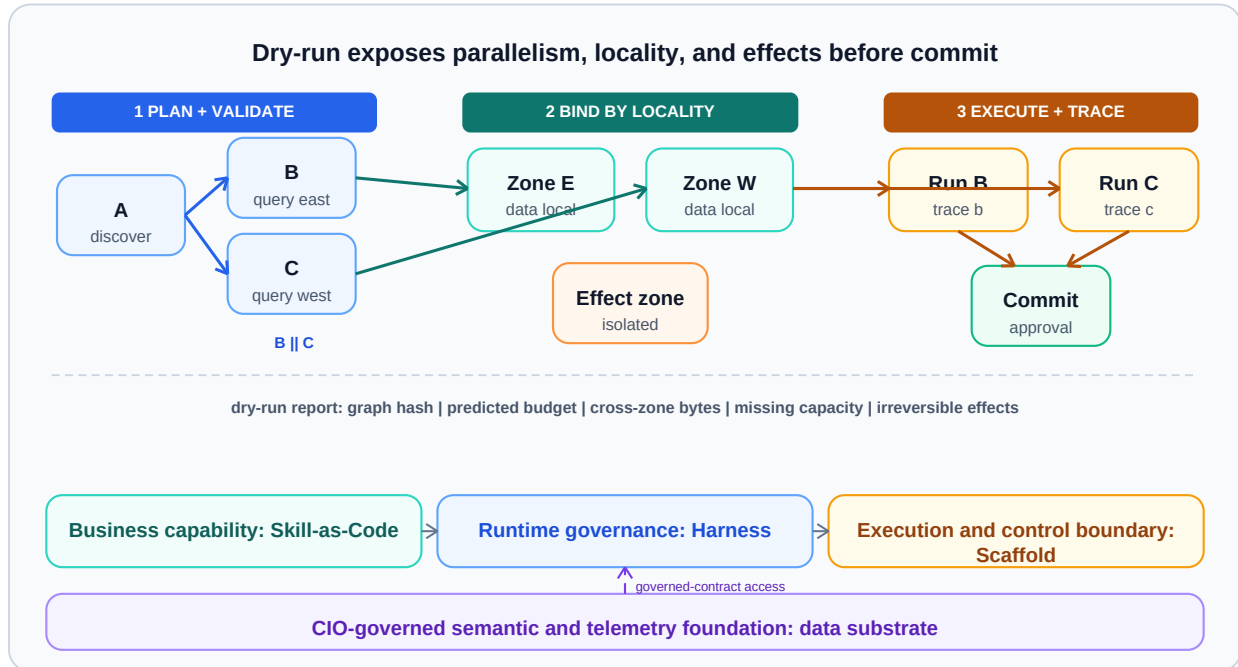


Figure 6. Planning-time dry-run and locality-aware execution. The Harness first validates and annotates the graph, then groups independent nodes near required data and binds effects to appropriate isolation zones. Explicit graphs permit concurrency and locality optimization while preserving a reviewable pre-execution decision point.

Figure 6 supports a modest claim: graph visibility creates an optimization opportunity; it does not guarantee a speedup. Dry-run adds planning overhead, and locality can conflict with load balance, policy, or accelerator availability. Evaluation must therefore report both avoided execution cost and dry-run cost.

8.4 Skill-as-Code

Treating a Skill as a prompt file makes capability growth difficult to review. Skill-as-Code applies a software lifecycle to the complete capability contract: source-controlled declarations, schema linting, effect review, contract tests, sandbox tests, model-version tests, signed release artifacts, staged rollout, monitoring, and rollback. That a Skill deserves the status of a software unit with its own identity, and that identity persistence across update, rollback, and removal is measurable rather than assumed, has since been developed as an ontology in its own right [5]; we take that framing as given. What this subsection adds is narrower and specific to a probabilistic control plane: the point in the lifecycle at which a converged Skill stops being regenerated and becomes frozen code, and the release gate that decides when it may.

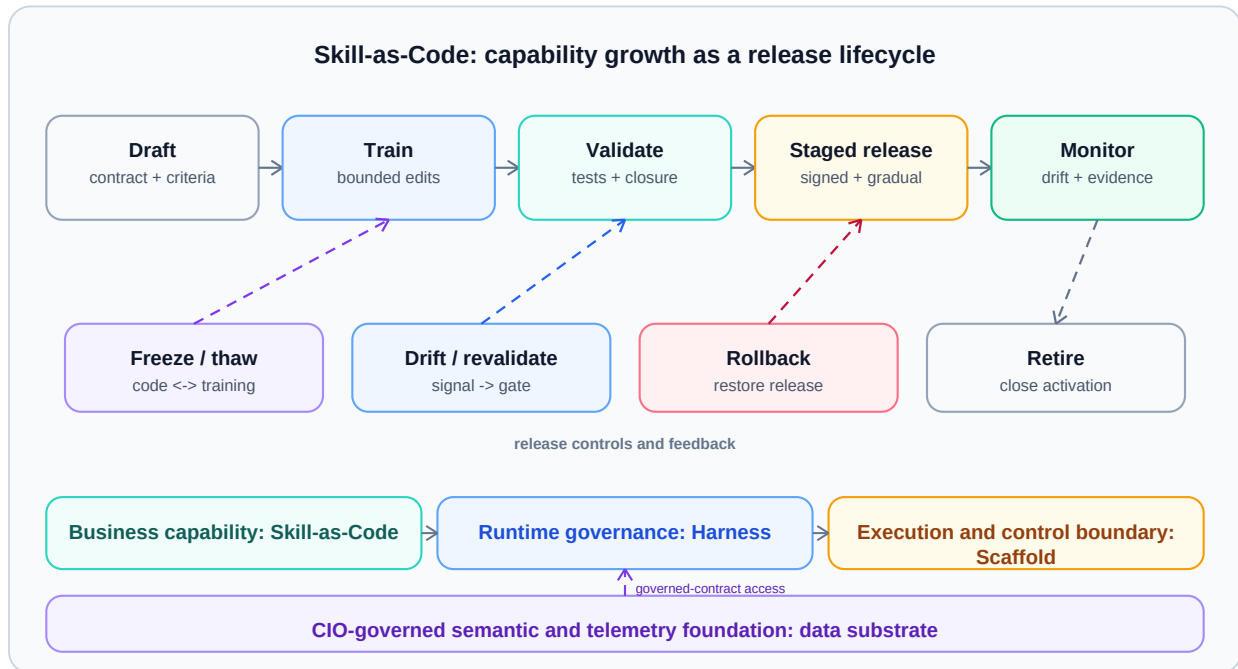


Figure 7. Skill-as-Code lifecycle. A candidate moves through draft, training, validation, staged release, and monitoring. Drift triggers revalidation; operators may freeze or thaw a Skill, roll back a staged or active release, and retire an identity while retaining lifecycle continuity and audit evidence.

Figure 7 makes validation and reversibility part of the release architecture rather than a documentation convention. The lifecycle is draft -> train -> validate -> staged release -> monitor; observed drift returns the Skill to revalidation, a thaw returns frozen behavior to training, rollback restores a prior staged release, and retirement closes activation while preserving identity history. Operation closure is a validation gate: tests must show that externally visible behavior is expressible through declared effects. Model-version stability is tested at the same boundary, and a release is blocked if contract adherence, path selection, or postcondition satisfaction regresses.

Skill-as-Code also serves as a deterministic anchor in the probabilistic control plane. A Skill's operation space closes once its tool-set is stable, its output format is structured, and its decision branches are enumerable; at that point the execution process can be frozen into code rather than regenerated probabilistically each turn. Frozen code removes model sampling from that execution path and embeds a stable procedure, but its execution is deterministic only for fixed inputs, dependency and external-service snapshots, random seed, and execution semantics. Under those conditions, the control plane has probabilistic planning, deterministic frozen-code execution, and deterministic gates. The frozen prefix of Section 7.1 stabilizes the context; the qualified frozen procedure stabilizes the decision within it. Together, they make the high-cohesion design law enforceable within the declared execution snapshot rather than claiming determinism

across mutable environments.

8.5 Training Before Freezing: A Dual-Subgoal Reward

Skill-as-Code explains how a stabilized Skill is governed and frozen, but not how the Skill reached stability in the first place. We treat the versioned Skill as trainable external state edited under a bounded loop with validation gates and a rejected-edit buffer. A single scalar $r(s)$ in $[0,1]$ is a lossy projection of a natural-language goal: it cannot tell the optimizer whether a failure lies in the outcome or in the process, nor which output sub-domain regressed. The following gate is therefore a design hypothesis to be compared against scalar and output-vector-only alternatives.

We propose to structure the reward along two subgoal dimensions, both derived from artifacts the contract architecture already maintains. The vector-valued gate below is a concrete design hypothesis that instantiates the dual-subgoal intuition, not a claim of logical necessity. The outcome subgoal scores the result separately along each sub-domain of the Skill's declared output schema output schema, with an independent annotated benchmark per sub-domain and a weighted aggregate:

$$r_out(s) = \text{sum over } d \text{ in } \text{dom}(\text{output schema}) \text{ of } w_d * r_d(\text{output_d}, \text{benchmark_d}).$$

The process subgoal supplies a dense, immediate signal about how the result was reached: whether the correct tools and data sources were invoked, whether intermediate reasoning satisfied the declared checking criteria, and whether each step gate passed:

$$r_proc(s) = (1/(\text{steps})) \text{sum over } k \text{ of indicator}[\text{step_k satisfies the tool/data/logic/gate criteria}].$$

Each mini-batch computes both subgoal families, and the validation gate becomes vector-valued rather than scalar: an edit is rejected when any protected output sub-domain regresses beyond tolerance or when protected process consistency drops, so the rejected-edit buffer records component-annotated negative feedback instead of an undifferentiated score.

The scalar $r_out(s)$ remains useful as a report statistic but is not the admission object. Let

$$R(s) = (r_d(s))_{d \text{ in } \text{dom}(\text{output schema})}, r_proc(s).$$

Before training, the evaluator fixes the verifier and tolerance ϵ_j for every component, together with protected components P and target components T . A proposed Skill s' is admitted relative to s only when

$$\text{Accept}(s'|s) = [\text{bigwedge}_j \text{ in } PR_j(s') \text{ at least } R_j(s) - \epsilon_j] \text{ and } [\text{bigvee}_j \text{ in } TR_j(s') > R_j(s) + \epsilon_j].$$

This Pareto-style gate prevents an aggregate gain from hiding a protected-domain regression while still requiring a material improvement on at least one declared target.

The role of the human changes accordingly: instead of judging every iteration, the human confirms the evaluation method once, deciding which benchmark scores each sub-domain and which criteria gate each step, after which reward computation is automatic. Training therefore decomposes naturally: different sub-agents, or per-sub-domain benchmarks, can guide the iteration of a complex Skill separately.

Relation to concurrent work on regression control.

Our decomposition indexes credit by the sub-domains of the declared output schema output schema, which exist in the contract before any run occurs. The practical consequence is that the reward dimensions are fixed by the same declaration that the Harness already gates and that Section 7.4 uses to compute the run-time satisfaction ratio, so training-time attribution and run-time completeness share one structure. Whether contract-derived dimensions improve discrimination, attribution, or convergence is exactly the question posed by P15. Independent support for keeping regression control inside an optimization loop comes from continual optimizer evaluation [10]; the vector gate is one proposed implementation, not a result established by that source.

The two stages then compose into one lifecycle. As training converges, subgoals whose evaluation logic and process criteria stop changing have closed operation spaces; they cross the closure threshold of Section 8.4 and enter the freeze-to-code stage. Training and compilation are two phases of the same Skill lifecycle, both governed by the Harness maintenance subsystem. We state the efficiency claim as a falsifiable hypothesis (P15) rather than a result: compared with a scalar gate and an output-vector-only gate, the output-plus-process vector should improve offline discrimination and attribution and improve online convergence without sacrificing protected components at equal rollout budget; Protocol 9.8 specifies separate offline and online comparisons.

Caveat: trustworthiness of process criteria.

Because `r_proc` depends on judging whether a step satisfied its criteria, a dimension-annotated rejection is only as trustworthy as the criterion that produced it. Process criteria must therefore be preregistered artifacts rather than model-generated at scoring time, and counterfactual no-violation episodes must test whether the gate invents a process failure.

9. Falsifiable Evaluation Protocols

The present work is architectural and has no controlled implementation results. We therefore specify experiments that can refute its claims. Each protocol separates the intervention from workload, model, policy, and hardware changes. Reported outcomes should include confidence intervals and failure distributions, not only averages.

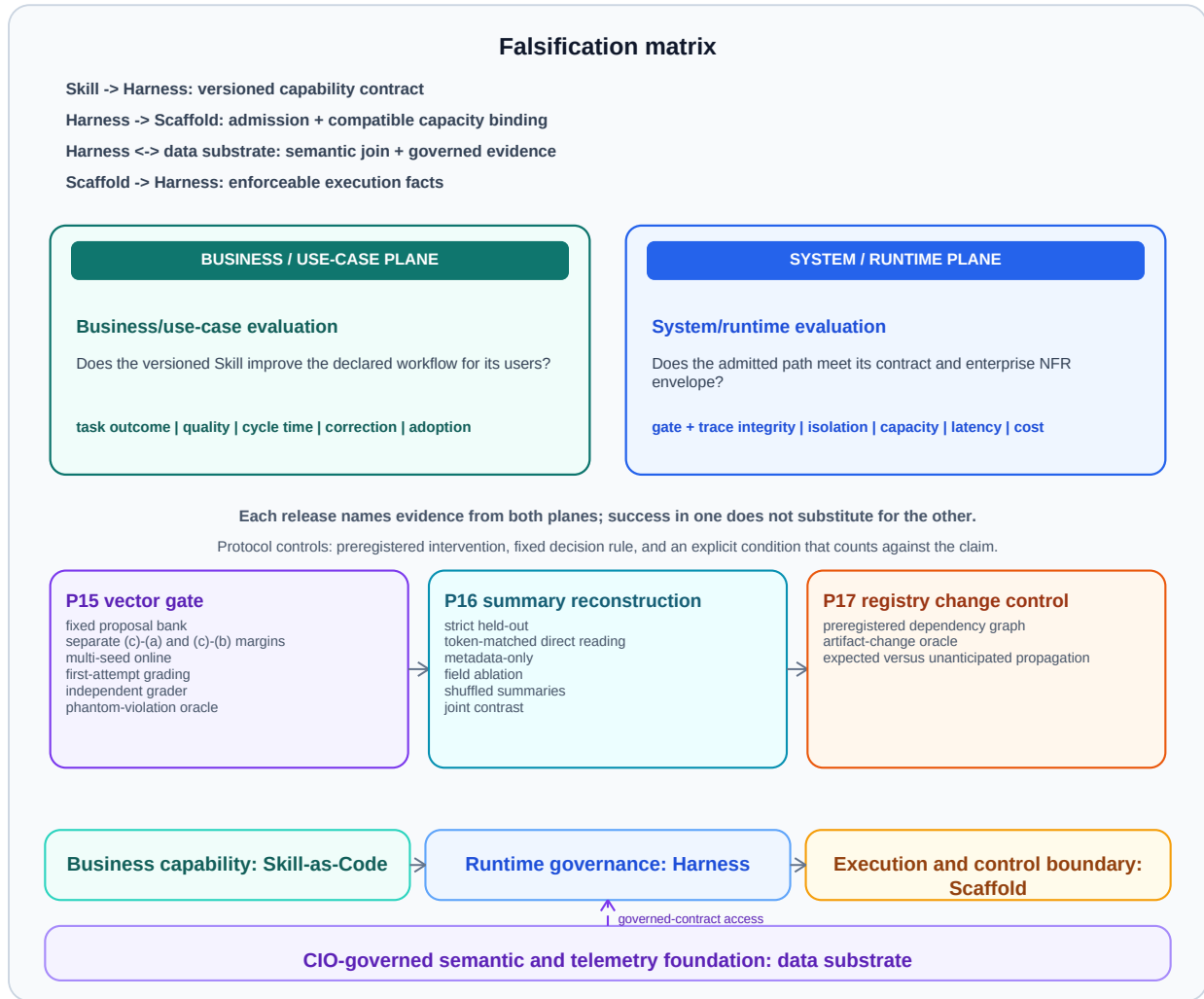


Figure 8. Evaluation matrix for the architecture. The P1 rows share a capability-count x Scaffold-count factorial design with injected recoupling factors. The remaining rows pair a boundary or subsystem hypothesis with a controlled intervention, observations, and a preregistered condition that counts against it.

Figure 8 summarizes the research program. Protocols 9.1 and 9.2 report one randomized capability-count x Scaffold-count factorial rather than two one-factor demonstrations. Before collection, declare the operating region Omega: workload family; model, tokenizer, Skill, Harness, and policy versions; external-service contracts; compatible Scaffold classes and topologies; and scheduler regime. The unit of randomization is one independent run of a preregistered workload instance under an assigned capability-count and Scaffold configuration. Randomize assignments within workload-instance and seed blocks, and repeat every cell over preregistered model or sampling seeds and independent runs. Before unblinding any outcome estimand, apply the six preregistered mechanism-level acceptance criteria to configuration, binding, information-flow, state-access, resource-channel, and scheduler-channel traces, then freeze and report each pass/fail decision. Outcome values cannot enter those decisions. Report cell estimates and interaction contrasts with confidence intervals and failure distributions.

The shared experiment has three estimands. Semantic invariance tests whether preregistered task metrics and the five interface semantics remain equivalent or non-inferior across compatible Scaffold conditions; equivalence or non-inferiority margins must be fixed before collection. Capacity response estimates the throughput and latency dose-response over compatible Scaffold count and topology at each capability count. Recoupling is the capability-count x Scaffold-count interaction in those semantic and capacity outcomes, judged against preregistered interaction margins. Separately randomized diagnostic arms inject shared-compute contention, locks, external-service quotas, authorization-backend delay, scheduler feedback, and mutable policy to localize departures. A confidence interval outside a semantic, capacity-response, or interaction margin counts against P1; a non-significant interaction alone is not

evidence of independence. The detailed protocols follow.

9.1 Shared Factorial Experiment: Capability Axis

Hypothesis. Within Omega, selective Skill activation keeps request-context growth and task interference bounded as unrelated Skills are added, and the capacity-response function for existing workloads remains invariant across capability-count rows.

Control. Execute all randomized factorial cells, treating capability count as the number of registered Skills presented to a fixed retrieval-and-activation mechanism and Scaffold count/topology as the compatible capacity dose. Keep active task Skills fixed and add unrelated Skills in randomized batches. Compare eager prompt loading with registry retrieval followed by typed activation, but analyze each activation strategy as a prespecified stratum rather than changing it across cells. Fix and report retrieval competence, run both single-source and many-source scales, and hold activation depth at one level unless depth is the preregistered intervention. These controls prevent changes in retrieval quality, corpus breadth, or routing depth from being mistaken for an effect of capability count.

Metrics. For semantic invariance, record task success, every-requirement satisfaction, plan validity, tool-selection errors, and the admission decision, activated path, authorized effects, binding constraints, and postconditions. For capacity response, record throughput and latency across every Scaffold column. Also report activated-Skill precision and recall, prompt tokens, policy-decision latency, and accidental exposure of control metadata. Aggregate recall is insufficient on its own because a small number of omitted requirements can invalidate an otherwise complete artifact. Report the fraction of runs in which every declared requirement was satisfied, alongside aggregate figures, and record which requirements were dropped rather than only how many.

Expected signature. Under selective activation, registry size grows while activated context, semantic outcomes, and capacity-response curves remain within their preregistered margins; eager loading may exhibit increasing context and cross-Skill confusion.

Falsification. The capability-axis claim is weakened if selective activation loses relevant Skills, its retrieval overhead dominates, semantic outcomes leave their equivalence or non-inferiority margins, or capacity-response contrasts vary across capability-count rows beyond the preregistered interaction margin.

9.2 Shared Factorial Experiment: Capacity and Interaction

Hypothesis. Within Omega, adding compatible Scaffold instances has a positive capacity response without changing preregistered semantic outcomes or the five interface semantics, and the response does not materially depend on capability count.

Control. Use the same randomized units, blocks, repeated seeds, runs, and factorial cells as Protocol 9.1. Vary only compatible Scaffold count and prespecified topology within each capability-count row; analyze homogeneous and locality-constrained pools as separate strata. Run each recoupling injection as a randomized diagnostic arm, preserving the base cell assignment and recording whether the injection changes a causal condition or leaves Omega.

Metrics. Estimate admission throughput, active-agent capacity, queueing delay, p50/p95/p99 latency, saturation, failure/retry rates, isolation violations, control-plane availability, cost per completed run, external-service backpressure, binding failures, and graph completion rate, each with confidence intervals where applicable. Report the capacity dose-response at preregistered points within the 0 to 100,000 admitted or active agents design target; this range is a measurement objective, not evidence that the implementation can sustain it. Test semantic equivalence or non-inferiority for task metrics and each interface semantic using their preregistered margins. Estimate capability-count x Scaffold-count interactions for both semantic outcomes and capacity-response parameters with confidence intervals rather than comparing only marginal means.

Expected signature. Queueing delay falls and throughput rises until a shared Harness, data, or external-service bottleneck is reached, while semantic outcomes and interface semantics remain inside their margins and capacity-response curves remain stable across capability-count rows.

Falsification. P1 is rejected in the tested Omega if all six preregistered mechanism-level condition criteria pass yet a compatible Scaffold intervention leaves a semantic equivalence or non-inferiority interval outside its margin, materially changes an interface semantic, produces a capacity-response departure beyond its margin, or yields a capability-count x Scaffold-count interaction outside its preregistered bound. Diagnostic-arm failures localize recoupling but do not by themselves show that the base operating region satisfies or violates P1.

9.3 Harness Bypass Ablation

Hypothesis. Complete Harness mediation reduces unauthorized effects and improves decision replay.

Control. Execute the same workloads with full mediation and with narrowly instrumented bypasses: direct tool invocation, untyped data fetch, unrecorded retry, and direct Scaffold selection.

Metrics. Count undeclared effects, policy violations, evidence omissions, non-replayable decisions, incident localization time, and false rejections.

Expected signature. Bypass variants may reduce local latency but produce more unexplained or policy-inconsistent runs.

Falsification. If bypass does not measurably affect control or replay under adversarial workloads, the claimed value of the Harness boundary is overstated or the test lacks meaningful effects.

9.4 Control-Plane Leakage

Hypothesis. Passing only activated contracts prevents sensitive or irrelevant registry state from entering request context.

Control. Seed the registry with canary metadata, deprecated schemas, and policy text that is never required by the workload. Compare eager loading, retrieved activation, and opaque-reference activation.

Metrics. Track canary reproduction, prompt tokens, deprecated-tool selection, policy-version confusion, and plan accuracy.

Falsification. The claim fails if control metadata appears in outputs or influences plans despite opaque activation, or if opacity prevents the planner from satisfying valid requests.

9.5 Path-Level Composition Safety

Hypothesis. Graph-level invariant checks catch hazards that per-Skill approval misses.

Control. Construct Skill pairs and longer paths that are benign individually but violate data-flow, privilege, or effect-ordering constraints when composed. Compare local allowlists with path-aware admission.

Metrics. Measure hazardous-path detection, false positives, dynamic-graph coverage, and decision latency.

Falsification. The claim is weakened if invariants cannot express realistic hazards, if dynamic insertion routinely escapes checks, or if false positives make valid composition impractical.

9.6 Dry-Run and Locality Economics

Hypothesis. Planning-time dry-run and locality-aware binding reduce wasted work or data movement enough to justify their overhead for complex graphs.

Control. Hold graph and Scaffold pool fixed. Compare immediate execution with dry-run admission, and locality-oblivious with locality-aware binding.

Metrics. Report dry-run latency, avoided failed-run cost, bytes moved across zones, completion latency, resource utilization, and policy rejection stage.

Falsification. The feature is not justified where dry-run predictions are poorly calibrated, planning cost exceeds avoided work, or locality choices reduce utilization enough to increase total cost.

9.7 Skill-as-Code Across Model Versions

Hypothesis. Contract and path tests identify model upgrades that alter Skill behavior before production release.

Control. Freeze Skill source and test corpus while varying planner and executor model versions. Include perturbations of input phrasing, tool errors, and partial data. Separate the component under test from the component that scores it, and grade the first attempt only. Without that separation an agent can silently repair itself mid-run and have the repaired output scored as a pass, which inflates pass rates toward a spurious ceiling [8]. Log every self-correction as an observation in its own right rather than crediting it.

Metrics. Measure schema adherence, declared-effect coverage, path stability, postcondition satisfaction, calibration of abstention, and test flakiness.

Falsification. The lifecycle is insufficient if production-significant changes pass the suite, or if nondeterminism makes the suite too unstable to gate releases.

9.8 Dual-Subgoal Reward for Skill Training

Hypothesis (P15). Under a natural-language goal, the output-plus-process vector gate admits more Pareto-valid edits and gives more valid attribution than scalar or output-vector-only gates, improving convergence at equal rollout budget without increasing protected-component regressions.

Offline gate comparison. Construct one fixed proposal bank for a multi-sub-domain task whose outputs combine, for example, numeric answers, textual summaries, and citations. Evaluate every proposal under the same baseline Skill with three gates: (a) one scalar; (b) the output vector (r_d_d in $\text{dom}(\text{output schema})$); and (c) the output-plus-process vector $R(s)$ of Section 8.5. Before scoring, preregister all verifiers, tolerances, protected and target sets, process criteria, decision thresholds, and the two comparator-specific contrasts (c)-(a) and (c)-(b). The offline P15 benefit claim requires both contrasts to meet their preregistered margins; averaging the comparators or passing only one contrast is insufficient. Independent experts or deterministic tests assign blinded oracle labels for Pareto validity, protected-component regression, and process-attribution truth without seeing gate decisions. Grade the executor's first attempt only, keep the grader separate from the executor, and log rather than credit self-corrections [8]. Include counterfactual proposals or episodes with no violation of a selected process criterion as a no-violation/phantom oracle.

Online training comparison. Run the same bounded Skill-training loop with each of the three gates. Hold model, workload distribution, optimizer prompt, rollout budget, verifier versions, and stopping rules fixed, but allow each loop to generate its own proposals. Run a matched multi-seed online comparison over preregistered random seeds. Preserve first-attempt grading and executor/grader separation. Preregister separate (c)-(a) and (c)-(b) contrasts for convergence, rollout cost, protected-component regressions, and attribution validity; both comparator-specific benefit contrasts must meet their margins. Compare convergence, accepted edits, rollout cost, protected-component regressions, and attribution validity, reporting seed-level distributions and confidence intervals.

Metrics. Offline, report gate discrimination for Pareto-valid versus dominated or regressive proposals, false admission and rejection rates, protected-regression detection, and attribution agreement with blinded oracle truth. Online, report edits or epochs to convergence, final per-domain and process scores, accepted edits, rollout cost, protected regressions, human interventions, and attribution validity.

Offline falsification. The discrimination and attribution part of P15 is weakened if configuration (c) fails a preregistered discrimination, protected-regression-detection, Pareto-valid-admission, or attribution-benefit margin against either (a) or (b), or reports process violations that the blinded phantom-violation oracle disproves.

Online falsification. The convergence and safety part of P15 is weakened if configuration (c) fails a preregistered convergence or rollout-cost benefit margin against either (a) or (b), increases protected-component regressions or degrades attribution validity relative to either comparator, or rejects enough Pareto-valid proposals that matched multi-seed training stalls.

9.9 Intermediate-Relation Decoupling and Reconstructive Reflection

Claims (P16, P17). P16 hypothesizes that a version-pinned Data Wiki summary is sufficient for a preregistered reconstruction use case on a strictly held-out report set. P17 proposes that registry schemas/modules remain independent under compliant one-sided changes while legitimate entry and IR updates remain observable.

P16 control. Preregister the use case, fields, report-structure descriptor, reconstruction metric, and acceptance margins before splitting reports into strict training, validation, and held-out sets. Build and revise summaries using training and validation only, then pin the summary and inspect the held-out set once. On identical held-out tasks, randomize the reconstruction agent to: (a) the pinned summary; (b) token-matched direct reading of source content; (c) metadata-only input; (d) preregistered field ablations of the summary; or (e) shuffled summaries assigned to the wrong source or report family. Deny source credentials and originals to every arm except the bounded direct-reading content supplied to arm (b), and pin the agent, verifier, token budget, and scoring procedure before one-shot held-out use.

P16 metrics and contrast. Report held-out reconstruction accuracy, field-level error, abstention, and confidence intervals for all pairwise preregistered contrasts. Summary-specific sufficiency requires the pinned summary to be equivalent or non-inferior to token-matched direct reading within its margin while exceeding metadata-only, shuffled-summary, and informative field-ablation controls by their positive margins. This joint contrast distinguishes summary structure from a generic context or token benefit.

P17 control and artifact-change oracle. Build a versioned Data Wiki and Theme Wiki over an enterprise-like corpus. Before each intervention, preregister an expected dependency graph for three change classes in each direction: (a) an entry-content update valid under the existing schema; (b) a new semantic type requiring a local registry schema/module update; and (c) a cross-registry use-case change expected to update the IR and named entries. A named artifact-change oracle records touched registry entries, schemas or modules, IR records, tests, and generated artifacts, then compares the observed change set with the necessary nodes and edges in the preregistered graph. Reviewers label extra and missing propagation while blinded to the implementation's claimed independence.

P17 metrics and falsification. Measure entry, IR, schema/module, test, and generated-artifact blast radius separately, and classify each observed change as required, missing, or unanticipated relative to the graph. P17 is weakened when a compliant one-sided entry or local schema/module intervention causes unanticipated schema/module propagation into the other registry. Expected entry, IR, test, and generated-artifact changes do not by themselves falsify independence.

P16 falsification. P16 is weakened when the pinned summary fails the joint summary-specific sufficiency contrast, when its one-shot held-out gap versus direct reading exceeds the preregistered margin, or when leakage auditing finds unauthorized source access, held-out-guided summary revision, or an unpinned verifier.

10. Discussion

10.1 Enterprise Adoption as Contract Adoption

The proposed architecture does not require four new departments. It requires four responsibility objects to remain distinguishable even when one organization owns several of them. A business product owner can change a Skill without acquiring authority over runtime policy; a Harness team can change admission and composition without silently redefining business meaning; a Scaffold team can evolve capacity while remaining accountable for performance, reliability, availability, observability, manageability, security, isolation, portability, elasticity, and capacity evolution; and the CIO-governed data substrate can evolve semantic and telemetry contracts on its own cadence. This separation gives architecture reviews a concrete agenda: owner, version, stable contract, evidence obligation, rollback authority, and permitted rate of change.

Adoption should therefore be evaluated on both planes in Table 4. Business/use-case evidence determines whether a Skill is worth releasing for a workflow. System/runtime evidence determines whether the activated path is governed and whether the Scaffold and external substrate satisfy the declared operational envelope. Neither success plane substitutes for the other, and a release process that reports only one leaves a different enterprise decision unanswered.

The Harness also creates a control-plane concentration risk. If its compiler, registry, or policy evaluator is unavailable, admission may stop; if one is compromised, a typed contract can become false assurance. Implementations need replicated control services, signed artifacts, independent attestation, and a declared degraded mode. New requests should fail closed when compilation is unavailable. Cached decisions require explicit freshness bounds, and registry failure may preserve already activated versions for in-flight work without admitting new ones. Degradation should reduce capability rather than widen governance.

Harness mediation and stack-external data access also have costs. Contract compilation, invariant evaluation, telemetry integration, semantic joins, and evidence retention consume latency, capacity, and administrative effort. Their cost must be measured against avoided failures and improved auditability; this paper provides no measured overhead. Selective activation can miss a relevant Skill, strict closure can reject useful exploration, path checking can be expensive for dynamic graphs, trace retention can conflict with privacy, and physical decoupling can fail under specialized hardware or residency constraints. The contracts make these trade-offs attributable; they do not remove them.

10.2 Open Adoption and Research Questions

Enterprise adoption raises organizational and technical questions together. What is the smallest contract language that captures effects without becoming a second programming language? Which decision rights and change cadences work when one team spans several responsibility objects? Which business metrics should block a Skill release, and which Scaffold NFR margins should block runtime admission? How should business and runtime telemetry be joined without transferring data ownership into the Harness? Which graph invariants can be checked statically, and which require runtime monitors? Which interventions preserve typed closure, complete mediation, effect non-interference, shared-state isolation, resource invariance, and scheduler independence inside a useful Omega? Which diagnostic injections localize recoupling, and at what workload complexity do dry-run and semantic-join costs become economical? These questions define the implementation and governance program that follows the architecture.

11. Limitations and Threats to Validity

This paper presents a reference architecture and proposed experiments, not a production system, enterprise adoption study, or empirical result. It does not demonstrate operation from 0 to 100,000 admitted or active agents. It does not establish business value, semantic-join quality, telemetry completeness, portability, safety, or any Scaffold NFR. It also does not show that the proposed ownership and change-cadence model resolves budget, incentive, procurement, or accountability conflicts across business units.

The responsibility mapping may underrepresent human approvals, long-running state, multi-tenant incentives, cross-organization trust, and regulated decision rights. Skill, Harness, and Scaffold are overloaded terms, and existing

systems may combine their responsibilities in one service. The analytical distinction still applies, but implementation may require refactoring and local role mapping. The stack-external data substrate is specified only at its contract boundary; its full on-policy access, lifecycle management, off-policy maintenance, and higher-order memory design remain outside scope.

The evidence base is temporally narrow. Apart from Conway's 1968 historical anchor, all retained contemporary sources were released in June or July 2026. They motivate governance, composition, lifecycle, regression control, and structured change, but they do not validate this complete architecture or its organizational thesis. Fast-moving preprints and artifacts may also change after the exact versions and dates recorded in the bibliography.

Conditional decoupling may hold only within a bounded operating region. A new Skill can require a new hardware class; a new Scaffold can expose timing or locality behavior that changes task quality. The six causal conditions may be incomplete, correlated, or difficult to hold fixed, while deterministic gates and trace identity can expose a violation without preventing it. Equivalence, non-inferiority, and interaction conclusions depend on chosen margins and repeated-run power; a non-significant interaction cannot establish independence. P15, P16, and P17 likewise remain hypotheses or propositions with preregistered controls, not observed results. Experiments must report where claims fail rather than classify couplings away.

12. Conclusion

Enterprise AI needs a contract across groups that change different things at different rates. This paper assigns reusable, versioned business capability and workflow assets to Skill; runtime compilation, admission, composition, governance, and evidence to Harness; the execution/control boundary and NFR ownership to Scaffold; and semantic joins plus integrated telemetry to a stack-external, independently CIO-governed data substrate. Architecture as a shared organizational contract is the paper's narrower thesis, not an external quotation or a claim that one organization chart fits every enterprise.

The scientific core remains the Testable Separability Conjecture. Inside a declared operating region, typed closure, complete mediation, effect non-interference, shared-state isolation, resource invariance, and scheduler independence are hypothesized to permit capability and compatible-capacity interventions to vary independently. Deterministic gates and trace identity make departures detectable, auditable, and attributable; they do not cause independence. The 0 to 100,000 admitted or active agents range is a measurable design target, not implementation evidence.

The paper therefore asks enterprises to demand two forms of proof. Business/use-case evidence must show that a Skill produces the intended workflow outcome. System/runtime evidence must show that the admitted path and its execution satisfy contract and NFR obligations. Neither substitutes for the other. P1 and the P15-P17 subsystem claims remain attached to explicit controls, estimands, margins, and falsification criteria. The next step is to implement the boundaries, run those protocols, and publish both the operating regions in which the contracts hold and the recoupling, organizational, or economic conditions under which they do not.

References

- [1] Melvin E. Conway. 1968-04. How Do Committees Invent?. *Datamation*. Volume 14, issue 4, pages 28-31. URL: https://www.melconway.com/Home/Committees_Paper.html.
- [2] Tallam, Krti. 2026-06-10. A Five-Plane Reference Architecture for Runtime Governance of Production AI Agents. arXiv preprint arXiv:2606.12320. URL: <https://arxiv.org/abs/2606.12320>.
- [3] Soni, Deepak. 2026-07-11. Falsifiable Release Gates for Self-Improving Systems: Standing Invariants at Scale. arXiv preprint arXiv:2607.13070. URL: <https://arxiv.org/abs/2607.13070>.
- [4] Xie, Yi, Du, Jiawei, Cheng, Yu, Zhou, Jiuan, Yin, Zhaoxia. 2026-06-13. Benign in Isolation, Harmful in Composition: Security Risks in Agent Skill Ecosystems. arXiv preprint arXiv:2606.15242. URL: <https://arxiv.org/abs/2606.15242>.
- [5] Fan, Haodi, Lan, Zucong. 2026-07-21. Skillware: A Software Ontology and Engineering Lifecycle for Persistent Behavioral Artifacts. arXiv preprint arXiv:2607.18970. URL: <https://arxiv.org/abs/2607.18970>.
- [6] Wang, Yanze, Yao, Pengfei, Sun, Tianyi, Hu, Chuanrui, Xiao, Yan, Han, Yunyun, Chen, Yifan, Sun, Jun, Deng, Yafeng. 2026-07-17. SkillCorpus: Consolidating and Evaluating the Open Skill Ecosystem for Real-World LLM Agents. arXiv preprint arXiv:2607.15557. URL: <https://arxiv.org/abs/2607.15557>.
- [7] Zhang, Chengjun, Gao, Yang, Hur, Jianna, Zhang, Jingjing, Samtani, Sagar. 2026-07-12. Cross-Layer Misalignment Detection in Agent Skills: A Progressive Loading-Aware Contrastive Learning Approach. arXiv preprint arXiv:2607.10534. URL: <https://arxiv.org/abs/2607.10534>.
- [8] Anand, Tejas Singh, Wang, Yuet Ying Christina, Jiang, Wanting, Masson, Steve, Zheng, Tian, Zhou, Bingjie. 2026-07-16. AEVAL: From Anecdotal to Deterministic Testing for Agentic Skill Workflows. arXiv preprint arXiv:2607.16345. URL: <https://arxiv.org/abs/2607.16345>.
- [9] Wang, Ruhan, Shi, Yucheng, Li, Zongxia, Li, Zhongzhi, Yu, Yue, Yang, Junyao, Panaganti, Kishan, Mi, Haitao, Zhou, Dongruo, Leoweiliang. 2026-07-14. Harness Handbook: Making Evolving Agent Harnesses Readable, Navigable, and Editable. arXiv preprint arXiv:2607.13285. URL: <https://arxiv.org/abs/2607.13285>.
- [10] Wang, Wenxiao, Kattakinda, Priyatham, Feizi, Soheil. 2026-07-15. Do Agent Optimizers Compound? A Continual-Learning Evaluation on Terminal-Bench 2.0. arXiv preprint arXiv:2607.14004. URL: <https://arxiv.org/abs/2607.14004>.
- [11] Hsu, Dan C., Lu, Luke. 2026-07-10. Scoped Verification for Reliable Long-Horizon Agentic Context Evolution under Distribution Shift. arXiv preprint arXiv:2607.09175. URL: <https://arxiv.org/abs/2607.09175>.
- [12] Tatavarthy, Venkat, Sharp, Beth, Suryanarayanan, Ganesh. 2026-06-04. Transforming ERP with agentic AI. AWS for SAP Blog. URL: <https://aws.amazon.com/blogs/awsfor sap/transforming-erp-with-agentic-ai/>.
- [13] Tzeveleka, Anastasia, Gruebler, Anna, Rodriguez, Antonio, Vitale, Sergio Garces, Tsakpinis, Aris. 2026-06-01. AgentOps: Operationalize agentic AI at scale with Amazon Bedrock AgentCore. AWS Machine Learning Blog. URL: <https://aws.amazon.com/blogs/machine-learning/agentops-operationalize-agentic-ai-at-scale-with-amazon-bedrock-agentcore/>.
- [14] Belson, Robert, Lolla, Aditya, Smith, Chris. 2026-06-22. Architecting distributed agentic AI workloads across AWS hybrid cloud services. AWS Infrastructure and Sustainability Blog. URL: <https://aws.amazon.com/blogs/infrastructure-sustainability/architecting-distributed-agentic-ai-workloads-across-aws-hybrid-cloud-services/>.